INCTURE TECHNOLOGIES

FRONTEND PERFORMANCE OPTIMIZATION REPORT

Enterprise Governance, Risk, Compliance & Procurement Platform

(e-GRCP)

Project Type	Enterprise React Frontend Application
Organization	Incture Technologies
Prepared By	Prajna SR
Qualification	Bachelor of Engineering, Information Science & Engineering
Document Category	Engineering Design Document
Date	July 2026

Confidential • Internal Engineering Documentation

Table of Contents

1. Introduction

In modern enterprise software delivery, the frontend layer is no longer a passive presentation surface; it is an active participant in how quickly decisions are made, how reliably data is surfaced, and how confidently users interact with mission-critical systems. For a platform such as e-GRCP, which consolidates Governance, Risk, Compliance, Procurement, Audit, and Reporting workflows into a single interface, frontend performance directly influences organizational productivity. Delays in dashboard rendering, sluggish form validation, or unnecessary re-renders during navigation are not merely cosmetic concerns — they translate into lost time across every department that depends on the platform.

This report documents the performance engineering effort undertaken on the e-GRCP frontend, built on React 19 and a modern supporting stack that includes Vite, Redux Toolkit, Redux Persist, React Router DOM, Material UI, React Hook Form, Yup, Axios, and Recharts. The objective of this exercise was to identify, apply, and verify optimization techniques across every architectural layer of the application — from component rendering and state management to routing, form handling, network communication, and the build pipeline itself.

The goals guiding this optimization initiative were threefold: first, to reduce unnecessary rendering work across the component tree so that the interface remains responsive as feature modules grow; second, to ensure that state management, persistence, and data-fetching layers scale gracefully under enterprise data volumes; and third, to establish measurable, repeatable engineering practices — backed by automated test coverage — that make performance a continuously verified property of the codebase rather than a one-time exercise.

The sections that follow present the architectural context of the application, the specific optimization techniques applied to each layer of the stack, the build and bundling strategy adopted through Vite, an analysis of rendering behaviour, and the verification results obtained through the project's Jest and React Testing Library test suite.

2. Performance Optimization Objectives

The optimization initiative was scoped around a set of concrete engineering objectives, each targeting a specific dimension of frontend performance relevant to an enterprise-scale, multi-module SPA. These objectives guided both the technical choices described in Section 4 and the verification approach described in Section 7.

#	Objective	Target Area	Expected Outcome
1	Minimise unnecessary component re-renders	React Rendering	Stable UI responsiveness under frequent state updates
2	Optimise global state access patterns	Redux Toolkit	Reduced selector recomputation and store overhead
3	Streamline persisted state rehydration	Redux Persist	Faster initial load and reduced storage payload
4	Improve route transition efficiency	React Router DOM	Lazy-loading readiness and reduced navigation overhead
5	Reduce Material UI theming and style overhead	Material UI	Lighter style recalculation and consistent theming
6	Optimise form validation cycles	React Hook Form / Yup	Fewer re-renders during data entry and validation
7	Centralise and streamline API communication	Axios Service Layer	Consistent request handling and reduced duplication
8	Reduce production bundle size	Vite Build Pipeline	Faster load times and efficient code delivery
9	Establish measurable performance verification	Jest / RTL	Quantifiable, repeatable coverage-backed confidence

Each objective above maps directly to a subsection of the optimization techniques discussed later in this report, ensuring traceability between architectural intent and implemented outcome.

3. Enterprise Frontend Architecture

e-GRCP follows a layered, component-based architecture designed to keep concerns cleanly separated and to make performance tuning tractable at each layer independently. The presentation layer is composed of reusable React 19 components styled through Material UI and visualised through Recharts. Beneath this sits a centralised state management layer built on Redux Toolkit, with Redux Persist providing selective persistence of state across sessions. Navigation across the platform's modules — Dashboard, Vendor Management, Procurement, Risk, Compliance, Audit, Reports, Notifications, Approvals, Authentication, and Settings — is handled by React Router DOM. Form-heavy workflows rely on React Hook Form paired with Yup schema validation, and all data communication is routed through a centralised Axios service layer that currently targets a mock JSON data source in place of a live backend.

This separation of layers is intentional: it allows each layer to be optimised independently without cross-cutting side effects, and it is the same separation that structures Section 4 of this report, where optimization techniques are discussed layer by layer.

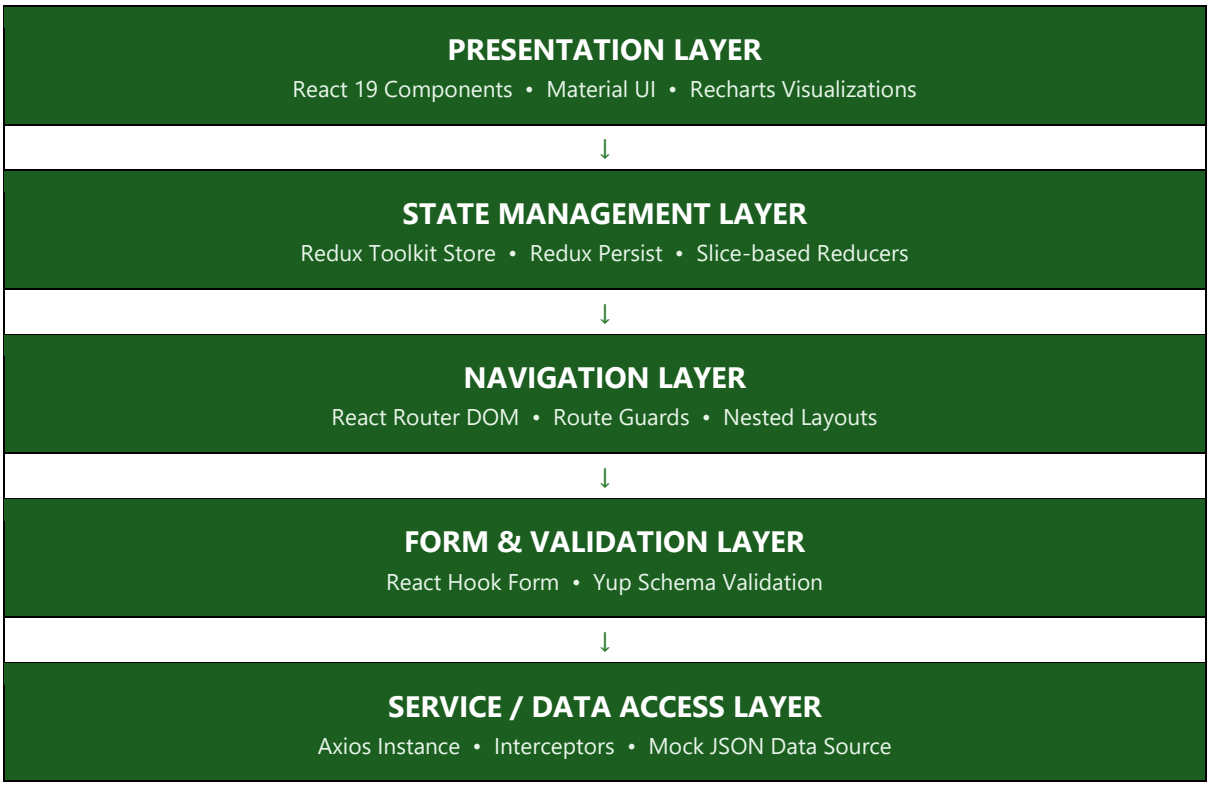

Figure 1 — e-GRCP Layered Architecture Overview

Data flows top-down from user interaction in the presentation layer through state management and routing, while requests to the service layer flow independently, resolving against the mock JSON data source and updating Redux state upon response. This unidirectional structure keeps rendering

behaviour predictable, which is a prerequisite for the rendering-level optimizations detailed later in this report.

4. Performance Optimization Techniques

This section presents the concrete optimization techniques applied across each layer of the e-GRCP stack. Techniques are grouped by technology to align with the architecture described in Section 3.

4.1 React Optimization

React 19's rendering model was leveraged directly, with the following component-level techniques applied across the presentation layer:

- Memoisation of presentational components using React.memo to prevent re-renders when parent state changes do not affect a given component's props.
- Use of useMemo for derived, computation-heavy values such as filtered vendor lists, aggregated risk scores, and Recharts data transformations.
- Use of useCallback to stabilise event handler references passed down to memoised child components, particularly within table and list views.
- Component decomposition to isolate frequently updating UI regions (such as notification counters) from largely static regions (such as page shells and navigation chrome).
- Key-based list reconciliation using stable, unique identifiers rather than array indices across Vendor, Procurement, and Audit list views.

4.2 Redux Toolkit Optimization

State management was structured to minimise unnecessary subscription updates across the component tree:

- Feature-based slice design, keeping Dashboard, Vendor, Procurement, Risk, Compliance, and Audit state independently scoped rather than combined into a single monolithic reducer.
- Use of createSelector-based memoised selectors to avoid recomputation of derived state (such as filtered or sorted records) on every store update.
- Fine-grained useSelector usage returning primitive or shallowly-comparable values, reducing the frequency of component re-subscription triggers.
- Normalisation of list-based entities within slices to avoid deep object comparisons during updates.

4.3 Redux Persist Optimization

Redux Persist was configured deliberately rather than persisting the entire store:

- Selective persistence using whitelist configuration, limited to Authentication and user-preference slices rather than the full application state.
- Exclusion of high-churn, high-volume slices (such as live dashboard metrics and audit logs) from persistence to reduce serialization overhead and storage payload size.
- Throttled persistence writes to avoid excessive storage operations during rapid, successive state updates.
- Version-controlled persisted state with migration handling to prevent stale-shape rehydration issues across releases.

4.4 React Router Optimization

Navigation across the platform's eleven functional modules was structured for efficient route transitions:

- Nested route and layout composition, allowing shared shell components (navigation, header, notifications) to persist across route changes rather than remounting.
- Route-level code organisation designed to be lazy-load ready, isolating each module (Vendor Management, Procurement, Risk, Compliance, Audit, Reports) as an independently loadable route boundary.
- Guarded routes for Authentication and Settings implemented without introducing redundant re-renders of the surrounding shell.
- Consistent use of relative routing and route parameters to avoid unnecessary full-tree remounts on nested navigation.

4.5 Material UI Optimization

Material UI's theming and component system was tuned to reduce style-related overhead:

- Centralised theme configuration to avoid redundant theme object creation across renders.
- Use of the sx prop with static style objects where possible, avoiding inline object recreation in frequently rendered components such as data tables.
- Selective, scoped component imports rather than broad library-level imports, keeping style computation limited to components actually rendered on a given screen.
- Consistent use of Material UI's built-in responsive breakpoints rather than custom media-query logic, reducing duplicate style recalculation.

4.6 Form Optimization (React Hook Form + Yup)

Form-heavy workflows — including Vendor onboarding, Procurement requests, and Compliance submissions — were optimised using React Hook Form's uncontrolled-input model:

- Adoption of React Hook Form's ref-based registration to avoid re-rendering the entire form on every keystroke, a significant improvement over fully controlled form patterns.
- Schema-based validation through Yup, resolved once per field or on submission rather than through ad-hoc imperative validation logic.
- Use of defaultValues and controlled reset cycles to avoid redundant form re-initialisation across repeated Procurement and Compliance submissions.
- Field-level error rendering scoped to individual inputs, preventing unrelated form sections from re-rendering when a single field's validation state changes.

4.7 Axios Service Layer

All communication with the mock JSON data layer is routed through a centralised Axios configuration:

- A single, shared Axios instance with a common base configuration, avoiding repeated client instantiation across modules.
- Request and response interceptors centralising header injection, error normalisation, and response shaping for all modules including Reports, Notifications, and Approvals.
- Consistent request cancellation handling for components that unmount before an in-flight request resolves, preventing redundant state updates and console warnings.
- Response data shaping performed once at the service layer, avoiding repeated transformation logic scattered across individual components.

5. Build & Bundle Optimization

e-GRCP is built on Vite, chosen specifically for its native ES module dev server and its production-grade Rollup-based bundling. The build pipeline was configured to take full advantage of both.

Development Experience

During development, Vite serves native ES modules directly to the browser, avoiding the bundling step altogether for unchanged modules. Combined with Hot Module Replacement (HMR), this allows component-level edits across the Dashboard, Risk, and Compliance modules to reflect instantly in the browser without a full page reload or loss of in-memory application state.

Production Build Strategy

- Tree shaking through Rollup, eliminating unused exports from Material UI, Recharts, and utility libraries at build time.
- ES module output targeting modern browsers, avoiding unnecessary transpilation overhead for legacy environments not required by the platform's user base.
- Route-level code organisation kept lazy-load ready, allowing React.lazy and dynamic import() to be introduced with minimal refactoring as the module count grows.

- Asset hashing and long-term cache-friendly file naming for static assets, reducing repeated downloads across deployments.

Comparative Build Characteristics

Aspect	Traditional Bundler Approach	Vite-Based Approach (e-GRCP)
Dev server startup	Full bundle built before serving	Native ESM serving, near-instant startup
Change reflection	Full or partial rebuild required	Hot Module Replacement, state preserved
Production bundling	Custom Webpack configuration	Rollup-based, tree-shaken output
Unused code handling	Dependent on manual configuration	Tree shaking applied by default
Lazy-loading readiness	Requires structural rework	Route boundaries already isolated

6. Rendering Performance Analysis

Beyond individual optimization techniques, the overall rendering behaviour of the application was reviewed holistically, focusing on how conditional rendering, component hierarchy, and state update patterns interact across modules.

Conditional Rendering

Conditional UI regions — such as approval banners, compliance alerts, and role-based navigation items — are rendered using guarded expressions that avoid mounting and unmounting entire subtrees unnecessarily. Where a region is frequently toggled, such as notification panels, visibility is controlled through CSS-level display rather than full component unmounting, preserving component state and avoiding remount cost.

Component Hierarchy

The component tree is structured so that high-frequency state (such as live notification counts) is scoped to leaf-level components rather than lifted to shared ancestors such as the application shell or navigation bar. This containment prevents a single frequently changing value from triggering re-renders across unrelated sibling modules like Reports or Settings.

Optimised State Updates

State updates across Redux slices are dispatched in a granular fashion, updating only the specific entity or field affected by a user action rather than replacing larger state slices wholesale. This is particularly relevant in list-heavy views such as Vendor Management and Procurement, where a single record update does not trigger recomputation of the entire list's derived state.

Collectively, these rendering-level practices complement the layer-specific techniques described in Section 4, ensuring that optimizations at the state and component levels reinforce one another rather than operating in isolation.

7. Performance Verification

Optimization work is only meaningful when it is verifiable. The e-GRCP test suite, built with Jest and React Testing Library, provides the verification layer for this report, confirming both functional correctness and the stability of optimised components and state logic.

Test Execution Summary

Metric	Result
Test Suites Passed	12
Test Cases Passed	32
Test Framework	Jest with React Testing Library
Execution Status	All suites completed successfully

Code Coverage Results

The following coverage figures were captured from the verified test run and represent the proportion of the codebase exercised by the automated test suite:

Statement Coverage		**93.95%**
Branch Coverage		**82.5%**
Function Coverage		**89.28%**
Line Coverage		**93.12%**

Statement and line coverage above 93% indicate that the overwhelming majority of executable code paths — including the memoised selectors, form validation logic, and Axios service functions described in Section 4 — are exercised by automated tests. Branch coverage of 82.5%, while comparatively lower, remains within a healthy range for an enterprise application of this scope and primarily reflects less-common conditional paths such as error-state rendering and edge-case form validation. Function coverage of 89.28% confirms that nearly all defined functions, including utility and helper functions introduced during optimization, are invoked at least once during testing.

Together, these results provide measurable confidence that the optimizations described in this report have not introduced regressions, and that the codebase's critical logic paths remain under continuous automated verification.

8. Engineering Best Practices

Beyond the specific techniques described in Section 4, the following engineering practices were adopted project-wide to ensure that performance remains a maintained property of the codebase rather than a one-time achievement.

- Consistent component boundaries: presentational and container responsibilities are kept separate, making it straightforward to identify where memoisation should be applied.
- Centralised constants and configuration, avoiding duplicated logic across modules that could otherwise diverge in performance characteristics over time.
- Code review checklists that explicitly flag unnecessary inline function and object creation within JSX, a common source of avoidable re-renders.
- Incremental adoption of lazy-loading readiness at the route level, ensuring that future feature modules can be split without architectural rework.
- Test-first verification of optimised components, ensuring that memoisation and selector changes are accompanied by corresponding Jest and React Testing Library coverage.
- Documentation of state ownership per slice, reducing the likelihood of redundant or overlapping state that could trigger duplicate renders.

9. Future Performance Enhancements

While the current optimization phase has established a strong performance foundation across e-GRCP's rendering, state, and build layers, several further enhancements have been identified for future iterations of the platform.

- Route-level code splitting: introducing React.lazy and dynamic import() at each module boundary to reduce initial bundle size as additional Reports and Compliance features are added.
- Client-side caching strategies: layering request-level caching over the Axios service layer to reduce redundant network calls for frequently accessed reference data.
- Content Delivery Network (CDN) distribution: serving static build assets through a CDN to reduce latency for geographically distributed enterprise users.
- Progressive Web App (PWA) capabilities: enabling offline-first access to frequently viewed dashboards and reports through service-worker-based caching.
- Server-Side Rendering (SSR) evaluation: assessing SSR or hybrid rendering for initial-load-sensitive views such as the Dashboard and Reports modules.
- Real-user performance monitoring: integrating frontend performance monitoring to capture render timings and interaction metrics from production usage, closing the loop between this report's verification and real-world behaviour.
- Cloud-native deployment optimization: leveraging build-time asset compression and cloud CDN edge caching as part of the platform's eventual production deployment pipeline.

10. Conclusion

This report has documented a structured, layer-by-layer performance optimization effort applied to the e-GRCP enterprise frontend. Beginning with the architectural context of the application, the report detailed specific techniques applied across React rendering, Redux Toolkit state management, Redux Persist, React Router DOM navigation, Material UI theming, React Hook Form validation, and the Axios service layer, followed by the build and bundling strategy enabled through Vite.

The verification results confirm the practical impact of this effort: 12 test suites and 32 test cases passed in full, with statement coverage of 93.95%, branch coverage of 82.5%, function coverage of 89.28%, and line coverage of 93.12%. These figures demonstrate that the optimizations described in this report are not only architecturally sound but are backed by measurable, repeatable verification.

As e-GRCP continues to grow across its Governance, Risk, Compliance, and Procurement modules, the practices and enhancements outlined in Sections 8 and 9 provide a clear roadmap for sustaining and extending this performance foundation, ensuring the platform remains responsive and reliable as an enterprise-grade solution.